FINANCIAL DATA ANALYTICS II

Interactive Portfolio Analytics Application

1. Project Overview

In this project, you will build an **interactive web application using Streamlit** that allows a user to construct and analyze equity portfolios in real time. The application will accept user inputs (ticker symbols, date ranges, and portfolio preferences), download data, and produce the return calculations, risk analytics, correlation analysis, and portfolio optimizations covered in Weeks 1 through 3 — all within a responsive, self-contained interface. You will also implement new analytical features — risk contribution decomposition, estimation window sensitivity analysis, and a custom portfolio builder — that go beyond notebook-based analysis.

This project counts toward the **projects component (40%) of your final course grade**. It is graded **pass/fail** (see Section 5). If you completed the extra-credit Project 1, you are encouraged to reuse any code or functions from that submission as a starting point.

Your final deliverable is a **live, publicly accessible application** deployed on Streamlit Community Cloud, backed by a **public GitHub repository** containing all source code. I will evaluate your project by using the app directly and reviewing the repository.

2. Application Requirements

Your application must implement **all** of the functionality described below. You have creative freedom in how you design the interface — but every requirement listed here must be present and functional. The app must be organized into **at least four distinct sections** using Streamlit's tabs (`st.tabs`), sidebar navigation, or multi-page layout. A single long scrolling page with no structure is not acceptable.

2.1 User Inputs and Data Retrieval

1. **Ticker entry.** The user enters between 3 and 10 stock ticker symbols. Validate that at least 3 and no more than 10 valid tickers are provided before proceeding.
2. **Date range selection.** The user selects a start and end date. Enforce a minimum range of 2 years.
3. **Risk-free rate input.** Provide a numeric input (defaulting to 2%) for the annualized risk-free rate used in Sharpe and Sortino ratio calculations.
4. **Data download.** Download adjusted closing prices for the selected tickers and for the S&P 500 benchmark (^GSPC) using `yfinance`. The S&P 500 serves as a benchmark for comparison only — do not include it as an asset in any portfolio optimization. If any ticker fails to download or returns insufficient data, display a clear error message identifying which ticker(s) caused the problem — do not crash silently.
5. **Handling partial data.** If a ticker has data for only part of the selected date range, handle it gracefully. The recommended approach is to truncate to the overlapping date range and inform the user. Alternatively, drop any ticker with more than 5% missing values and warn the user. Do not silently proceed with misaligned data or crash with a raw error.

Caching is required. Use `@st.cache_data` to cache downloaded data and expensive calculations. An app that re-downloads data on every widget interaction will be unacceptably slow.

2.2 Return Computation and Exploratory Analysis

1. A **summary statistics table** showing annualized mean return, annualized volatility, skewness, kurtosis, minimum daily return, and maximum daily return for each stock and the benchmark.
2. A **cumulative wealth index chart** showing the growth of $10,000 in each stock and the S&P 500. All series on a single chart with a legend. The user should be able to select/deselect individual stocks using a multi-select widget or checkboxes.
3. A **distribution plot** for a user-selected stock with a toggle between two views: (a) a histogram of daily returns overlaid with a fitted normal distribution curve, and (b) a Q-Q plot comparing the return distribution to a theoretical normal. The Q-Q plot is more effective than a histogram at revealing fat tails. You can generate Q-Q data using `scipy.stats.probplot`.

2.3 Risk Analysis

1. **Rolling volatility.** A chart of rolling annualized volatility for each stock. The user must be able to adjust the window length (e.g., 30, 60, 90, or 120 days) using a slider or select box.
2. **Drawdown analysis.** A drawdown chart for a user-selected stock showing the percentage drawdown from the running peak over time. Display the maximum drawdown value prominently (e.g., as an `st.metric` widget).
3. **Risk-adjusted metrics table.** Sharpe ratio and Sortino ratio for each stock and the benchmark, computed using the user-specified risk-free rate.

2.4 Correlation and Covariance Analysis

1. **Correlation heatmap.** A heatmap of the pairwise correlation matrix of daily returns. Values annotated on cells. Diverging color scale centered at zero.
2. **Rolling correlation.** Let the user select any two stocks and a rolling window length, and plot the rolling correlation over time.
3. **Covariance matrix.** Display the covariance matrix of daily returns in a table. This can be a secondary view (e.g., behind an expander or toggle).

2.5 Portfolio Construction and Optimization

This is the most important section of the application.

Equal-Weight Portfolio

- Compute and display annualized return, annualized volatility, Sharpe ratio, Sortino ratio, and maximum drawdown for the 1/N portfolio.

Optimized Portfolios

- Compute the **Global Minimum Variance (GMV)** and **Maximum Sharpe Ratio (Tangency)** portfolios using `scipy.optimize.minimize` with no-short-selling constraints (weights between 0 and 1, sum to 1). Display optimal weights, annualized return, volatility, Sharpe ratio, Sortino ratio, and maximum drawdown for each.
- Display portfolio weights using a **bar chart or pie chart**.

Risk Contribution (New)

- For the GMV and tangency portfolios, compute and display the **percentage risk contribution (PRC)** for each asset as a bar chart. The PRC for asset *i* is:

$$PRC_i = w_i \cdot (\Sigma w)_i \,/\, \sigma^2_p$$

  The PRC values sum to 1 and represent each position's fraction of total portfolio risk. Display PRC alongside the portfolio weights so the user can see when a stock contributes more (or less) risk than its weight suggests. The app should include a brief description near this chart explaining what risk contribution means — for example, that a stock with 10% weight but 25% risk contribution is a disproportionate source of portfolio volatility.

Efficient Frontier

- Generate and plot the efficient frontier in mean–standard deviation space. On the same chart, mark the equal-weight portfolio, GMV portfolio, tangency portfolio, custom portfolio (see below), each individual stock, and the S&P 500 benchmark. Draw the Capital Allocation Line from the risk-free rate through the tangency portfolio. Include a brief description near the chart explaining what the efficient frontier and CAL represent.

Custom Portfolio (New)

- Let the user **manually set portfolio weights** using sliders (one per stock, defaulting to equal weight). Since sliders are independent, **normalize** by dividing each value by their sum, and display the normalized weights prominently. As the user adjusts sliders, dynamically update the custom portfolio's annualized return, volatility, Sharpe ratio, Sortino ratio, and maximum drawdown. The custom portfolio must also appear as a labeled point on the efficient frontier chart.

Portfolio Comparison

- A **cumulative wealth index chart** comparing the equal-weight, GMV, tangency, and custom portfolios alongside the S&P 500.
- A **summary comparison table** with annualized return, volatility, Sharpe ratio, Sortino ratio, and maximum drawdown for all portfolios and the benchmark.

2.6 Estimation Window Sensitivity (New)

Mean-variance optimization is sensitive to its inputs — small changes in estimated returns or covariances can produce dramatically different portfolio weights. This section lets the user see that directly.

Allow the user to select the **lookback period** used to compute the mean returns and covariance matrix that feed into the optimizer. Provide at least three options (e.g., trailing 1 year, 3 years, 5 years, and the full sample), constrained by the user's selected date range — do not offer a 5-year lookback if the user selected only 3 years of data.

For each lookback window, display a **comparison table** showing:

1. **GMV portfolio** — weights, annualized return, and volatility.
2. **Tangency portfolio** — weights, annualized return, volatility, and Sharpe ratio.

You may also include a grouped bar chart of the weights across windows. Include a brief note in the interface explaining why this matters — that historical optimization results are only as stable as the inputs that produced them.

3. Application Design and Usability

Your app does not need to be beautiful, but it must be functional, organized, and not confusing.

1. **Layout.** At least four distinct sections using sidebar, tabs, or multiple pages. Separate input configuration from analysis output, and group related analyses together.
2. **Headings and labels.** Every chart must have a title and labeled axes. Every table must have a header.
3. **Loading states.** Use `st.spinner` during computation or downloads. The user should never stare at a frozen screen.
4. **Error handling.** The app must not crash with a raw Python traceback for common user mistakes (invalid tickers, empty inputs, bad date ranges). Display user-friendly error messages using `st.error` or `st.warning`. **This will be tested** — I will enter invalid tickers and nonsensical inputs to check.
5. **About / Methodology section.** Include a section (sidebar expander, dedicated tab, or collapsible section) that briefly explains the analytical methods, key assumptions (risk-free rate convention, annualization, simple returns), and data source.

4. Deliverables

You must submit exactly two things:

4.1 Deployed Application URL

The public URL of your application, deployed on **Streamlit Community Cloud**. The app must be live and accessible at the time of grading.

4.2 Public GitHub Repository

The URL of a **public GitHub repository** containing:

- Your application code (`app.py`, or multiple `.py` files if you use a multi-file structure).
- A `requirements.txt` in the root directory listing all required Python packages, one per line. Version pins are optional. Other files produced by `uv` (such as `pyproject.toml` and `uv.lock`) can be included.
- A `.gitignore` file that excludes `.venv/`, `__pycache__/`, `*.pyc`, `.python-version`, `.DS_Store`, `Thumbs.db`, and `.streamlit/`.

Do not commit your `.venv/` directory or data files. A `README.md` is welcome but not required.

5. Grading

5.1 Pass/Fail Evaluation

This project is graded **pass/fail** and counts toward the projects component (40%) of your final course grade.

A passing app meets the following criteria:

- The deployed app is live and accessible.
- All functionality described in Sections 2.1 through 2.6 is present, working, and producing correct results.
- The app handles bad inputs (invalid tickers, nonsensical date ranges) gracefully without crashing.
- The GitHub repository is public and contains the application code and `requirements.txt`.

Common reasons for a failing grade:

- The deployed link is broken or the app fails to load.
- A required feature from Sections 2.1 through 2.6 is missing or non-functional.
- The app crashes with a Python traceback during normal use or error-handling tests.
- Calculations are materially incorrect (e.g., wrong Sharpe ratio formula, incorrect portfolio variance, broken optimization).
- The GitHub repository is not public or does not contain the application code.

5.2 How the App Will Be Tested

1. **Multiple ticker combinations** will be tested, including well-known stocks, less common tickers, and deliberately invalid tickers. Valid tickers should produce correct results; invalid tickers should produce clear error messages without crashing.
2. **Different date ranges** will be tested, including short ranges near the 2-year minimum and long ranges spanning 10+ years.
3. **Every feature** described in Sections 2.1 through 2.6 will be checked for presence, functionality, and correctness.
4. **The GitHub repository** will be reviewed to verify it is public and contains source code matching the deployed app.

6. Policies

6.1 Academic Integrity

This is an individual project. You may not submit an application that is substantially similar to another student's. You may reuse your own code from Project 1 freely.

6.2 Use of AI Tools

You are expected and encouraged to use AI tools (ChatGPT, Claude, GitHub Copilot, Gemini, or any others) as extensively as you like. The only thing that matters is the final product: your deployed app must work, every required feature must be present, and it must handle testing without crashing.

6.3 Late Submissions

Late submissions will not be accepted under any circumstances.

7. Technical Reminders

7.1 Financial Calculations

These conventions must be followed consistently throughout your application:

- **Use simple (arithmetic) returns** for all portfolio-level calculations including the covariance matrix, portfolio optimization, and efficient frontier. Log returns are not additive across assets and will produce incorrect portfolio weights.
- **Risk-free rate.** The user provides an annualized rate (default 2%). Convert to a daily rate by dividing by 252 where needed. Use this consistently for all Sharpe and Sortino ratio calculations.
- **Annualization.** For daily data, multiply mean daily return by 252 for annualized return; multiply daily standard deviation by $\sqrt{252}$ for annualized volatility.
- **Cumulative returns.** Use `(1 + r).cumprod()` on simple returns. A common mistake is forgetting to add 1, or accidentally using log returns with this formula.
- **Portfolio variance.** Use the full quadratic form $w^T\Sigma w$ with the covariance matrix. Do not simply take the weighted average of individual variances.
- **Sortino ratio.** The denominator is downside deviation (computed using only returns below the risk-free rate), not total standard deviation. Using total standard deviation gives the Sharpe ratio, not the Sortino.
- **Efficient frontier.** Solve a constrained optimization at each target return level. Random portfolio simulation is not the efficient frontier.
- **Tangency portfolio.** Formulate as minimizing the negative Sharpe ratio (since `scipy.optimize.minimize` minimizes by default).
- **Optimization constraints.** No-short-selling bounds should be `(0, 1)` for each weight, with a sum-to-one equality constraint: `{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}`.
- **Rolling calculations** produce NaN for the first window − 1 observations. This is expected, not an error.
- **Use adjusted close prices** to account for dividends and splits.

7.2 Streamlit and Deployment

- **Streamlit reruns.** Streamlit reruns your entire script top to bottom every time a widget value changes. This is why `@st.cache_data` is essential. Cache any function that downloads data or performs optimization.
- **Slider normalization.** Streamlit sliders are independent widgets. Normalize custom weights by dividing each value by the sum of all values. Display the normalized weights so the user sees the actual allocation.
- **Optimization failures.** `scipy.optimize.minimize` does not always converge. Wrap optimization calls in error handling and display a user-friendly message if the optimizer fails. Check `result.success` before using the weights.
- **yfinance reliability.** Wrap data downloads in `try`/`except` blocks. Use `@st.cache_data` with a `ttl` parameter (e.g., `ttl=3600`) to prevent excessive re-fetching.
- **Risk contribution formula.** In NumPy: `(weights * (cov_matrix @ weights)) / portfolio_variance`. The PRC values should sum to 1 (minor floating-point deviations are fine).
- **Plotting.** Streamlit supports Matplotlib, Plotly, and Altair. Plotly is recommended for interactivity but Matplotlib is fine. All plots must have titles and axis labels.
- **Performance.** Use `st.spinner(...)` or a progress bar for any computation that takes more than a few seconds.
- **Deployment.** Push your code and `requirements.txt` to a public GitHub repo, then connect it at share.streamlit.io. Test the deployed app thoroughly before submitting — it may behave differently from your local version if dependencies or Python versions differ.
- **Resource limits.** The free tier has a ~1 GB memory limit. Avoid unnecessary copies of large DataFrames and ensure `@st.cache_data` is applied everywhere it should be.
- **App hibernation.** Free-tier apps sleep after inactivity and take a minute or two to wake up. This is expected — I will wait. However, if your app fails to load even after waiting, that is a problem. Test your link periodically.

8. Getting Started

8.1 Prerequisites

You need the following tools installed:

- **uv** — Python package and project manager. Also handles installing Python itself.
- **GitHub Desktop** (recommended) or the **GitHub CLI (`gh`)** — for creating repositories and pushing code to GitHub. Either works; use whichever you are more comfortable with.

To install GitHub Desktop:

```
winget install GitHub.GitHubDesktop   # Windows
brew install --cask github            # macOS
```

If you prefer the command line, use the GitHub CLI as covered in the *Building & Deploying a Stock Analysis App with Streamlit* guide.

- A **GitHub account** and a free **Streamlit Community Cloud** account at streamlit.io/cloud, linked to your GitHub.

8.2 Project Setup

Create a new project (do not build on top of the guide's `stock-app`):

```
mkdir portfolio-app
cd portfolio-app
uv init
```

Delete the starter file (`hello.py` or `main.py`), then install dependencies:

```
uv add streamlit yfinance pandas plotly scipy numpy statsmodels
```

If you need additional packages later, run `uv add <package>` locally **and** add the package name to your `requirements.txt` so Streamlit Cloud installs it during deployment.

Create three files in the project folder:

- `app.py` — your application code.
- `requirements.txt` — list every third-party package your code imports (e.g., `streamlit`, `yfinance`, `pandas`, `plotly`, `scipy`, `numpy`, `statsmodels`), one per line. Standard library modules (like `datetime`) do not need to be listed.
- `.gitignore` — should contain:

```
.venv/
__pycache__/
*.pyc
.python-version
.DS_Store
Thumbs.db
.streamlit/
```

Run locally with: `uv run streamlit run app.py`

8.3 Git, GitHub, and Deployment

You need to create a **public** GitHub repository and connect it to Streamlit Community Cloud.

Using GitHub Desktop:

1. Open GitHub Desktop. Go to **File** → **Add Local Repository** and select your `portfolio-app` folder. GitHub Desktop will notice it is not yet a Git repository and offer to **create a repository** — click that link and confirm.
2. Your files will appear as changes. Write a commit message (e.g., "Initial commit") and click **Commit to main**.
3. Click **Publish repository**. Uncheck "Keep this code private" so the repository is public.
4. Each time you make changes, return to GitHub Desktop — your changes appear automatically. Write a message, click **Commit to main**, then **Push origin**.

Using the command line:

```
git init
git branch -M main
git add .
git commit -m "Initial commit"
gh repo create portfolio-app --public --source=. --remote=origin --push
```

Push subsequent updates with `git add .`, `git commit -m "description"`, `git push`.

Deploy to Streamlit Community Cloud:

1. Go to share.streamlit.io and sign in with your GitHub account.
2. Click **New app**, select your repository, branch (`main`), and main file (`app.py`).
3. Click **Deploy**. Streamlit will install your dependencies and launch the app at a public URL.
4. Each time you push to GitHub, the deployed app updates automatically.

8.4 Development Workflow

Build incrementally. Get the data download working first, then add one analytical section at a time. Do not try to build the entire app in one sitting.

Deploy early. As soon as you have a basic working version, push it and deploy. Do not wait until the last day — deployment issues are common and much easier to solve when you are not under deadline pressure.

Use AI tools aggressively throughout this process.